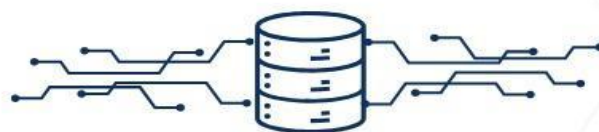
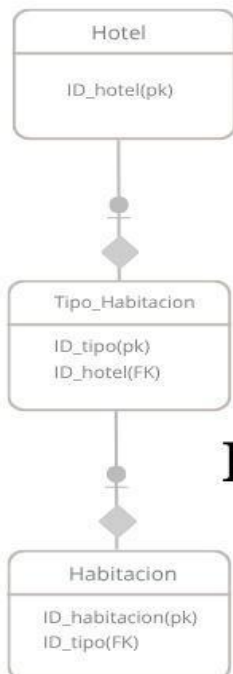


UNIVERSIDAD CENTROAMERICANA "JOSE SIMEON CAÑAS"



facultad de Ingenieria y arquitectura
Departamento de informática e electrónica

PROYECTO DE BASE DE DATOS SISTEMA DE GESTIÓN HOTELERA



Asignatura

Bases de Datos

Ciclo

Ciclo 01/2026

Catedratico

Ing. James Edward Humberstone Morales

Integrantes

Gary Geovanni Sibrian Deras	00021925
Herberth Ariel Tobar Sermeño	00158525
Vinicio Alonso Sibrian Vargas	00302725
Victor José Tamayo Hernández	00087025
Mauricio Ernesto Hernandez Marroquin	00013525

Equipo

Grupo-D



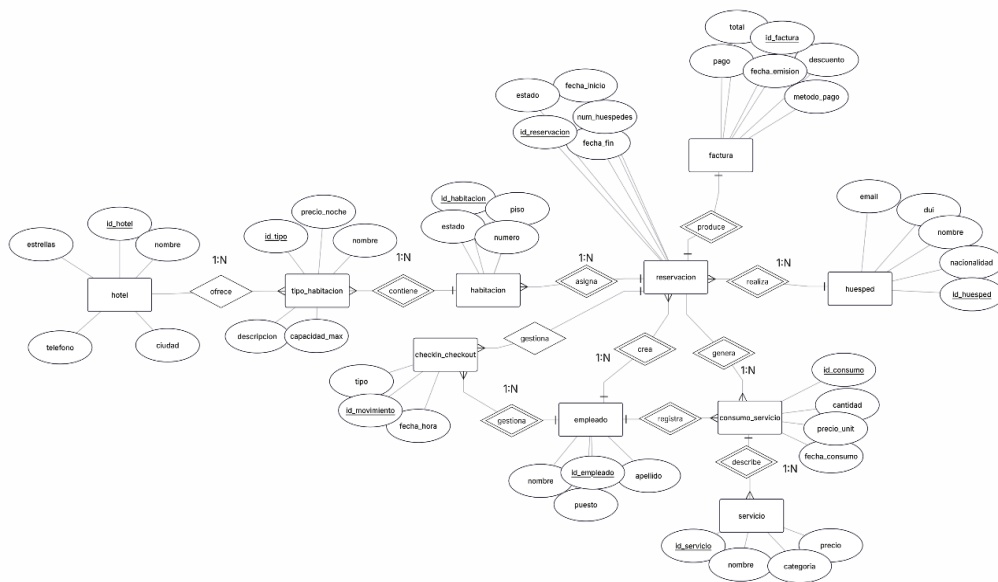
Descripción del escenario

El presente proyecto modela un sistema de gestión para el Hotel La Palma, un establecimiento de tres estrellas ubicado en San Salvador. El hotel cuenta con habitaciones de tipo estándar, doble y suite junior; ofrece servicios básicos de restaurante, lavandería y llamadas telefónicas. El sistema permite administrar las reservaciones de huéspedes, el control de check-in/check-out realizado por empleados, el registro de consumos de servicios adicionales y la generación automática de facturas al momento del checkout.

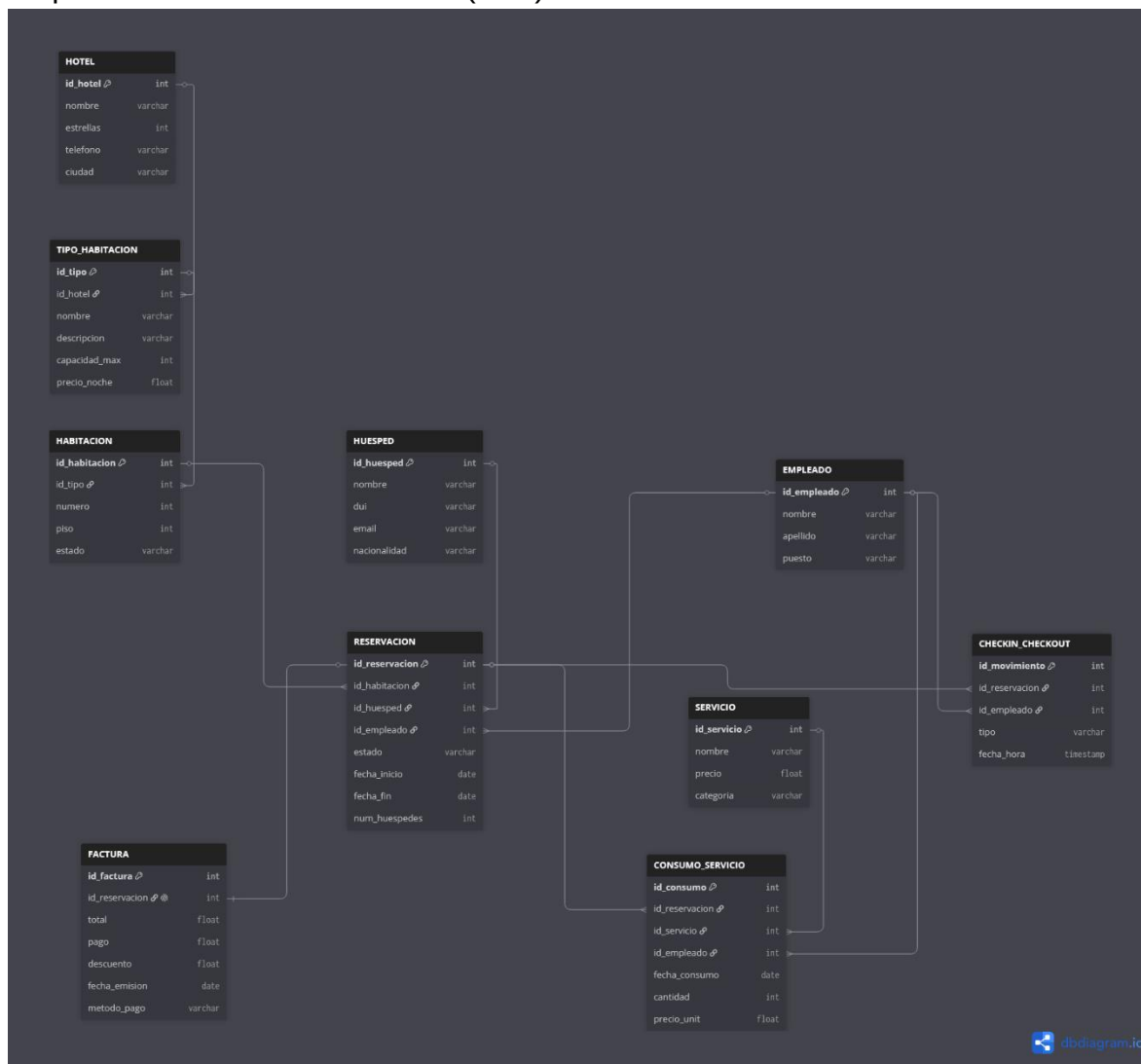
Regla del negocio

- Un huésped puede realizar múltiples reservaciones a lo largo del tiempo, pero no puede tener dos reservaciones activas para la misma habitación en el mismo período de fechas.
- Cada reservación cubre exactamente una habitación, para un rango de fechas determinado (fecha_inicio, fecha_fin).
- Una habitación pertenece a un tipo de habitación, y ese tipo está asociado a un hotel específico.
- Durante la estancia, el huésped puede consumir servicios adicionales (restaurante, lavandería, teléfono). Cada consumo queda registrado con fecha, cantidad y precio unitario vigente.
- Al realizar el check-out, se genera una factura consolidada que incluye el costo de la habitación por noches y el total de servicios consumidos. No puede existir más de una factura por reservación.
- Un empleado puede gestionar múltiples movimientos de check-in y check-out, y también puede registrar consumos de servicios.
- El estado de una habitación puede ser: disponible, ocupada o en mantenimiento.
- El estado de una reservación puede ser: pendiente, activa, completada o cancelada.

diagrama Entidad–Relación en notación Chen



esquema relacional normalizado (3FN)



Diccionario de datos:

Hotel

Campo	Tipo	Restricción	Nulo	Descripción
id_hotel	INT	PK	NO	Identificador único del hotel
nombre	VARCHAR	NOT NULL	NO	Nombre comercial del hotel
estrellas	INT	CHECK(1-5)	NO	Categoría del hotel en estrellas
telefono	VARCHAR	-	SÍ	Número de contacto del hotel
ciudad	VARCHAR	NOT NULL	NO	Ciudad donde se ubica el hotel

Tipo habitacion

Campo	Tipo	Restricción	Nulo	Descripción
id_tipo	INT	PK	NO	Identificador único del tipo de habitación
id_hotel	INT	FK	NO	Hotel al que pertenece este tipo
nombre	VARCHAR	NOT NULL	NO	Ej: Estándar, Doble, Suite Junior
descripcion	VARCHAR	-	SÍ	Descripción de amenidades del tipo
capacidad_max	INT	NOT NULL	NO	Número máximo de huéspedes permitidos
precio_noche	FLOAT	NOT NULL	NO	Precio base por noche en USD

Habitación

Campo	Tipo	Restricción	Nulo	Descripción
id_habitacion	INT	PK	NO	Identificador único de la habitación
id_tipo	INT	FK	NO	Tipo al que pertenece la habitación
numero	INT	NOT NULL	NO	Número de habitación en el edificio
piso	INT	NOT NULL	NO	Piso donde se ubica la habitación
estado	VARCHAR	NOT NULL	NO	disponible ocupada en mantenimiento

Huésped

Campo	Tipo	Restricción	Nulo	Descripción
id_huesped	INT	PK	NO	Identificador único del huésped
nombre	VARCHAR	NOT NULL	NO	Nombre completo del huésped
dui	VARCHAR	UNIQUE	SÍ	Documento de identidad (huéspedes locales)
email	VARCHAR	UNIQUE	SÍ	Correo electrónico de contacto
nacionalidad	VARCHAR	-	SÍ	País de origen del huésped

Empleado

Campo	Tipo	Restricción	Nulo	Descripción
id_empleado	INT	PK	NO	Identificador único del empleado
nombre	VARCHAR	NOT NULL	NO	Nombre del empleado
apellido	VARCHAR	NOT NULL	NO	Apellido del empleado
puesto	VARCHAR	NOT NULL	NO	Ej: recepcionista, mesero, camarero

Reservación

Campo	Tipo	Restricción	Nulo	Descripción
id_reservacion	INT	PK	NO	Identificador único de la reservación
id_habitacion	INT	FK	NO	Habitación reservada
id_huesped	INT	FK	NO	Huésped que realiza la reservación
id_empleado	INT	FK	NO	Empleado que registra la reservación
estado	VARCHAR	NOT NULL	NO	pendiente activa completada cancelada
fecha_inicio	DATE	NOT NULL	NO	Fecha de inicio de la estancia
fecha_fin	DATE	NOT NULL	NO	Fecha de salida planificada
num_huespedes	INT	NOT NULL	NO	Cantidad de huéspedes en la reservación

Factura

Campo	Tipo	Restricción	Nulo	Descripción
id_factura	INT	PK	NO	Identificador único de la factura
id_reservacion	INT	FK, UNIQUE	NO	Reservación asociada (relación 1:1)
total	FLOAT	NOT NULL	NO	Monto total (habitación + servicios)
pago	FLOAT	NOT NULL	NO	Monto pagado por el huésped
descuento	FLOAT	DEFAULT 0	SÍ	Descuento aplicado en USD
fecha_emision	DATE	NOT NULL	NO	Fecha en que se generó la factura
metodo_pago	VARCHAR	NOT NULL	NO	efectivo tarjeta transferencia

CHECKIN_CHECKOUT

Campo	Tipo	Restricción	Nulo	Descripción
id_movimiento	INT	PK	NO	Identificador único del movimiento
id_reservacion	INT	FK	SÍ	Reservación asociada
id_empleado	INT	FK	SÍ	Empleado responsable
fecha_hora	TIMESTAMP	-	SÍ	Fecha y hora del movimiento
tipo	VARCHAR	-	SÍ	Tipo de movimiento (check-in o check-out)

Servicio

Campo	Tipo	Restricción	Nulo	Descripción
id_servicio	INT	PK	NO	Identificador único del servicio
nombre	VARCHAR	NOT NULL	NO	Ej: Desayuno, Lavandería, Llamada
precio	FLOAT	NOT NULL	NO	Precio estándar por unidad en USD
categoria	VARCHAR	NOT NULL	NO	restaurante lavanderia telefonia

Consumo servicio

Campo	Tipo	Restricción	Nulo	Descripción
id_consumo	INT	PK	NO	Identificador único autoincremental e independiente de cada consumo
id_reservacion	INT	FK	NO	Reservación en la que se consumió el servicio
id_servicio	INT	FK	NO	Servicio consumido
id_empleado	INT	FK	NO	Empleado que registró el consumo
fecha_consumo	DATE	-	NO	Fecha del consumo
cantidad	INT	NOT NULL	NO	Unidades consumidas
precio_unit	FLOAT	NOT NULL	NO	Precio unitario al momento del consumo

Documentación de Consultas Frecuentes

Verificación de registros por tabla

Objetivo

Verificar que los datos hayan sido cargados correctamente en todas las tablas principales de la base de datos.

Descripción

Esta consulta realiza un conteo de registros para cada tabla del sistema hotelero. Su propósito es validar la integridad de la carga inicial de datos y comprobar que cada entidad contiene información registrada.

Código

```
SELECT 'HOTEL' AS tabla, COUNT(*) AS registros FROM "HOTEL"
UNION ALL SELECT 'TIPO_HABITACION', COUNT(*) FROM "TIPO_HABITACION"
UNION ALL SELECT 'HABITACION', COUNT(*) FROM "HABITACION"
UNION ALL SELECT 'HUESPED', COUNT(*) FROM "HUESPED"
UNION ALL SELECT 'EMPLEADO', COUNT(*) FROM "EMPLEADO"
UNION ALL SELECT 'SERVICIO', COUNT(*) FROM "SERVICIO"
UNION ALL SELECT 'RESERVACION', COUNT(*) FROM "RESERVACION"
UNION ALL SELECT 'CHECKIN_CHECKOUT', COUNT(*) FROM "CHECKIN_CHECKOUT"
UNION ALL SELECT 'CONSUMO_SERVICIO', COUNT(*) FROM "CONSUMO_SERVICIO"
UNION ALL SELECT 'FACTURA', COUNT(*) FROM "FACTURA";
```

Resultado esperado

Muestra una lista con el nombre de cada tabla y la cantidad total de registros almacenados en ella.

Habitaciones disponibles para reservación

Objetivo

Identificar las habitaciones que se encuentran disponibles para ser reservadas dentro de un período específico.

Descripción

La consulta muestra únicamente aquellas habitaciones que no poseen reservaciones activas o pendientes dentro del rango de fechas seleccionado. Además, presenta información sobre ubicación, tipo, capacidad máxima y precio por noche.

Código

```
SELECT h.id_habitacion,
       CONCAT('Piso ', h.piso, ' - Hab. ', h.numero) AS ubicacion,
       UPPER(t.nombre) AS tipo,
       t.capacidad_max AS capacidad,
       CONCAT('$', t.precio_noche) AS precio_por_noche
FROM "HABITACION" h
JOIN "TIPO_HABITACION" t ON h.id_tipo = t.id_tipo
WHERE h.estado = 'disponible'
      AND h.id_habitacion NOT IN (
        SELECT id_habitacion FROM "RESERVACION"
        WHERE estado IN ('pendiente', 'activa')
              AND fecha_inicio < '2026-07-15'
              AND fecha_fin    > '2026-07-10'
      )
ORDER BY t.precio_noche ASC;
```

Resultado esperado

Genera un listado de habitaciones disponibles indicando ubicación, tipo de habitación, capacidad máxima y precio por noche.

Top 10 huéspedes con mayor gasto

Objetivo

Identificar los huéspedes que generan mayores ingresos para el hotel.

Descripción

La consulta calcula la cantidad de reservaciones realizadas por cada huésped, el gasto total acumulado y el promedio gastado por estancia. Los resultados se ordenan de forma descendente según el monto total gastado.

Código

```
SELECT h.id_huesped,  
       UPPER(CONCAT(h.nombre, ' (' , h.nacionalidad, ')')) AS huesped,  
       h.email,  
       COUNT(r.id_reservacion) AS total_reservaciones,  
       SUM(f.total)           AS gasto_total,  
       ROUND(AVG(f.total)::numeric, 2) AS promedio_por_estancia  
FROM "HUESPED" h  
JOIN "RESERVACION" r ON h.id_huesped = r.id_huesped  
JOIN "FACTURA" f     ON r.id_reservacion = f.id_reservacion  
GROUP BY h.id_huesped, h.nombre, h.nacionalidad, h.email  
ORDER BY gasto_total DESC  
LIMIT 10;
```

Resultado esperado

Muestra los diez huéspedes con mayor gasto acumulado, indicando correo electrónico, cantidad de reservaciones realizadas y promedio de gasto por estancia.

Servicios más consumidos por tipo de habitación

Objetivo

Analizar el consumo de servicios adicionales según el tipo de habitación reservada.

Descripción

La consulta relaciona los consumos registrados con los servicios utilizados y el tipo de habitación asociado a cada reservación. Esto permite identificar cuáles servicios presentan mayor demanda y cuáles generan mayores ingresos.

Código

```
SELECT UPPER(t.nombre) AS tipo_habitacion,
       UPPER(s.nombre) AS servicio,
       s.categoria,
       SUM(cs.cantidad) AS total_consumido,
       ROUND(SUM(cs.cantidad * cs.precio_unit)::numeric, 2) AS ingreso_generado
FROM "CONSUMO_SERVICIO" cs
JOIN "SERVICIO" s ON cs.id_servicio = s.id_servicio
JOIN "RESERVACION" r ON cs.id_reservacion = r.id_reservacion
JOIN "HABITACION" h ON r.id_habitacion = h.id_habitacion
JOIN "TIPO_HABITACION" t ON h.id_tipo = t.id_tipo
GROUP BY t.nombre, s.nombre, s.categoria
ORDER BY t.nombre, total_consumido DESC;
```

Resultado esperado

Presenta la cantidad total consumida de cada servicio y los ingresos generados, agrupados por tipo de habitación.

Tasa de ocupación por tipo de habitación

Objetivo

Determinar el porcentaje de ocupación de cada categoría de habitación.

Descripción

La consulta calcula la cantidad de reservaciones registradas por mes para cada tipo de habitación y genera una tasa de ocupación basada en el número total de habitaciones disponibles para dicha categoría.

Código

```
SELECT UPPER(t.nombre) AS tipo,
       TO_CHAR(r.fecha_inicio, 'Month YYYY') AS mes,
       EXTRACT(YEAR FROM r.fecha_inicio) AS anio,
       EXTRACT(MONTH FROM r.fecha_inicio) AS nro_mes,
       COUNT(r.id_reservacion) AS reservaciones_mes,
       (SELECT COUNT(*) FROM "HABITACION" WHERE id_tipo = t.id_tipo) AS total_habitaciones,
       ROUND(
         COUNT(r.id_reservacion) * 100.0 /
         NULLIF((SELECT COUNT(*) FROM "HABITACION" WHERE id_tipo = t.id_tipo), 0)::numeric,
         2) AS tasa_ocupacion_pct
FROM "RESERVACION" r
JOIN "HABITACION" h ON r.id_habitacion = h.id_habitacion
JOIN "TIPO_HABITACION" t ON h.id_tipo = t.id_tipo
WHERE r.estado IN ('activa', 'completada')
GROUP BY t.nombre, t.id_tipo, mes, anio, nro_mes
ORDER BY anio, nro_mes, t.nombre;
```

Resultado esperado

Muestra la cantidad de reservaciones registradas y el porcentaje de ocupación correspondiente a cada tipo de habitación.

Reporte mensual de ingresos

-Objetivo

Analizar el comportamiento financiero del hotel durante el año en curso.

Descripción

La consulta genera un resumen mensual de facturación, incluyendo cantidad de facturas emitidas, ingresos brutos, descuentos aplicados, ingresos netos, valor máximo facturado y ticket promedio.

Código

```
SELECT TO_CHAR(f.fecha_emision, 'Month')           AS mes,
       EXTRACT(MONTH FROM f.fecha_emision)         AS nro_mes,
       COUNT(f.id_factura)                         AS facturas_emitidas,
       SUM(f.total)                               AS ingresos_brutos,
       SUM(f.descuento)                           AS total_descuentos,
       ROUND((SUM(f.total) - SUM(f.descuento))::numeric, 2) AS ingresos_netos,
       MAX(f.total)                               AS factura_mas_alta,
       ROUND(AVG(f.total)::numeric, 2)             AS ticket_promedio
FROM "FACTURA" f
WHERE EXTRACT(YEAR FROM f.fecha_emision) = EXTRACT(YEAR FROM CURRENT_DATE)
GROUP BY mes, nro_mes
ORDER BY nro_mes;
```

Resultado esperado

Genera un reporte mensual que permite evaluar los ingresos obtenidos por el hotel, los descuentos otorgados y el comportamiento general de la facturación.

Documentación de Funciones y Disparadores

Función verificar_disponibilidad()

Objetivo

Validar que una habitación no tenga reservaciones activas o pendientes que se superpongan con las fechas de una nueva reservación o de una modificación de reservación existente.

Tipo

Función PL/pgSQL asociada a un trigger.

Valor de retorno

TRIGGER

Funcionamiento

La función verifica si existe alguna reservación activa o pendiente para la misma habitación dentro del rango de fechas especificado. En caso de detectar una superposición de fechas, genera una excepción y evita que la operación se complete.

Esta validación garantiza que una misma habitación no pueda ser asignada simultáneamente a diferentes huéspedes durante el mismo período.

Código

```
CREATE OR REPLACE FUNCTION verificar_disponibilidad()
RETURNS TRIGGER AS $$
BEGIN
    -- Verificar solapamiento con reservaciones activas/pendientes
    IF EXISTS (
        SELECT 1 FROM "RESERVACION"
        WHERE id_habitacion = NEW.id_habitacion
          AND estado IN ('pendiente', 'activa')
          AND id_reservacion <> NEW.id_reservacion -- excluir la propia fila en UPDATE
          AND fecha_inicio < NEW.fecha_fin
          AND fecha_fin    > NEW.fecha_inicio
    ) THEN
        RAISE EXCEPTION
        'Conflicto: la habitación % ya está reservada entre % y %.',
        NEW.id_habitacion,
        TO_CHAR(NEW.fecha_inicio, 'DD/MM/YYYY'),
        TO_CHAR(NEW.fecha_fin,    'DD/MM/YYYY');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Beneficio

Permite mantener la integridad de las reservaciones y evita conflictos derivados de asignar una misma habitación a diferentes huéspedes durante fechas coincidentes.

Trigger trg_verificar_disponibilidad

Objetivo

Garantizar que las reservaciones registradas en el sistema cumplan con la regla de negocio que impide reservar una habitación durante un período ya ocupado.

Tabla afectada

RESERVACION

Evento

BEFORE INSERT OR UPDATE

Funcionamiento

El trigger se ejecuta automáticamente antes de insertar o actualizar registros en la tabla RESERVACION. Su función es invocar la función verificar_disponibilidad() para validar la disponibilidad de la habitación seleccionada.

Si la validación detecta un conflicto de fechas, la operación es cancelada y se muestra un mensaje de error al usuario.

Código

```
CREATE TRIGGER trg_verificar_disponibilidad
BEFORE INSERT OR UPDATE ON "RESERVACION"
FOR EACH ROW EXECUTE FUNCTION verificar_disponibilidad();
```

Beneficio

Automatiza el control de disponibilidad de habitaciones, evitando errores humanos y asegurando el cumplimiento de las reglas de negocio definidas para el sistema hotelero.

Pruebas de Funcionamiento

Caso 1: Inserción válida

Descripción

Se registra una reservación para una habitación que no posee conflictos de fechas con otras reservaciones activas o pendientes.

Código

```
-- Caso exitoso (no hay conflicto):  
INSERT INTO "RESERVACION"(id_habitacion, id_huesped, id_empleado,  
    estado, fecha_inicio, fecha_fin, num_huespedes)  
VALUES (1, 1, 1, 'pendiente', '2026-07-01', '2026-07-05', 2);  
-- → Se inserta correctamente
```

Resultado esperado

La reservación se registra correctamente en la base de datos.

Caso 2: Inserción con conflicto de fechas

Descripción

Se intenta registrar una reservación para una habitación que ya posee una reservación activa o pendiente dentro del mismo rango de fechas.

Código

```
-- Caso de error (fechas que chocan):  
INSERT INTO "RESERVACION"(id_habitacion, id_huesped, id_empleado,  
    estado, fecha_inicio, fecha_fin, num_huespedes)  
VALUES (1, 2, 1, 'pendiente', '2026-07-03', '2026-07-08', 1);  
-- → ERROR: Conflicto: la habitación 1 ya está reservada entre 01/07/2026 y 05/07/2026
```

Resultado esperado

El sistema rechaza la operación y genera el siguiente mensaje:

ERROR: Conflicto: la habitación 1 ya está reservada entre 01/07/2026 y 05/07/2026

Lo anterior demuestra que el trigger funciona correctamente al impedir la creación de reservaciones con fechas superpuestas para una misma habitación.

Conclusión

El desarrollo del Sistema de Reservas para el Hotel La Palma permitió aplicar de manera integral los conceptos fundamentales del diseño y la administración de bases de datos relacionales, desde el modelado conceptual hasta la implementación de mecanismos de control automatizado.

1. Modelado de datos coherente con el negocio.

El diagrama Entidad–Relación en notación Chen y su posterior traducción al esquema relacional normalizado en Tercera Forma Normal (3FN) permitieron representar fielmente las reglas del negocio del escenario hotelero, evitando redundancias y garantizando que cada atributo dependiera únicamente de la clave primaria de su entidad. Esto se reflejó, por ejemplo, en la separación entre HOTEL, TIPO_HABITACION y HABITACION, que evita duplicar información de capacidad o precio en cada habitación individual.

2. Integridad referencial y de negocio.

La definición de relaciones como la de uno a uno entre RESERVACION y FACTURA, o la de uno a muchos entre HUESPED y RESERVACION, asegura que el modelo respete restricciones críticas del dominio, como el hecho de que no puede existir más de una factura por reservación o que una habitación pertenece a un único tipo y hotel.

3. Automatización de reglas de negocio mediante funciones y triggers.

La implementación de la función verificar_disponibilidad() junto con el trigger trg_verificar_disponibilidad demostró que es posible trasladar reglas de negocio complejas —como la prohibición de reservar una misma habitación en fechas superpuestas— directamente al motor de base de datos, reduciendo la dependencia de validaciones realizadas en la capa de aplicación y disminuyendo el riesgo de inconsistencias provocadas por error humano.

4. Valor analítico de las consultas implementadas.

Las consultas frecuentes diseñadas (disponibilidad de habitaciones, top de huéspedes por gasto, consumo de servicios por tipo de habitación, tasa de ocupación y reporte mensual de ingresos) evidencian que el modelo no solo soporta las operaciones transaccionales del día a día, sino que también permite generar información gerencial útil para la toma de decisiones, como identificar los servicios más rentables o los periodos de mayor ocupación.

5. Validación práctica del diseño.

Las pruebas de funcionamiento, tanto en el caso de inserción válida como en el caso de conflicto de fechas, confirmaron que el sistema responde de forma esperada ante escenarios reales de uso, rechazando operaciones que violarían la integridad del negocio y aceptando aquellas que cumplen con las reglas establecidas.

En conjunto, el proyecto demuestra que un diseño cuidadoso desde la fase conceptual, sumado al uso adecuado de normalización, restricciones de integridad y lógica procedural en PL/pgSQL, resulta en un sistema de base de datos robusto, escalable y alineado con las necesidades reales de gestión de un hotel de tres estrellas como La Palma.